

LiteDoc: A Browser-Native Document Layout Analysis Engine with Evolutionary Hyperparameter Optimization

0xovo

LiteDoc Project — Open Source Document Processing

contact@litedoc.xyz

Abstract — We present LiteDoc, a fully browser-native document layout analysis (DLA) engine that extracts structured Markdown from arbitrary PDF files without server round-trips. Built on PDF.js and Tesseract.js WebAssembly, LiteDoc operates entirely within a single HTML file deployment. Reading order determination employs Kahn's topological sort on a directed acyclic graph constructed from geometric bounding box relationships, eliminating column interleaving without natural language processing or machine learning. Mathematical equations are detected through a Unicode symbol density heuristic covering ~90 mathematical symbols, while corrupted text is identified via a suspicion ratio scoring mechanism that analyzes Private Use Area characters, CJK blocks in Latin documents, and mixed-script penalties. The engine's 13 hyperparameters are tunable through an automated evolutionary optimization framework using genetic algorithms with holdout validation. Across a matched 77-PDF A/B comparison spanning 7 document categories, the bug-fixed release (v2.6.0) achieves a +4.8% improvement in average extraction score over the v2.5.2 baseline (52.1% → 56.9%), with pass rate ($\geq 70\%$ threshold) rising from 26% to 36.4%. The GA optimization framework is built; full tuning results are deferred to v3.0. LiteDoc is released under AGPL-3.0 and processes document data locally, ensuring data privacy for sensitive documents.

Keywords — *document layout analysis, PDF extraction, topological sorting, genetic algorithms, browser-native, optical character recognition*

1. INTRODUCTION

The extraction of structured, machine-readable text from Portable Document Format (PDF) files remains a persistent challenge in document processing. PDFs encode content as low-level drawing instructions — character glyphs, paths, and images — with no inherent notion of paragraphs, columns, tables, or reading order. Recovering the logical document structure from this "primordial soup" of coordinates requires sophisticated layout analysis.

Existing solutions fall into three categories. Server-side libraries such as pdfplumber, Camelot, and Tabula (in Python) provide programmatic extraction but require backend infrastructure and network transmission of potentially sensitive documents. Academic tools

like GROBID and Science Parse offer domain-specific extraction for scientific literature but are complex to deploy and maintain. Commercial APIs including Adobe Extract, Amazon Textract, and Senseforth deliver high accuracy but introduce vendor lock-in, per-document costs, and privacy concerns through server-side processing.

LiteDoc addresses these limitations by performing document layout analysis entirely within the browser. The engine combines Mozilla's PDF.js for PDF parsing, Tesseract.js compiled to WebAssembly for optical character recognition of scanned pages, and a suite of statistical heuristics for reading order determination, equation detection, gibberish filtering, table reconstruction, and vector figure extraction. The entire system is distributed as a

single HTML file with inlined CDN dependencies, deployable by opening a file or serving from any static host. All document data remains on the client device — no page content ever leaves the user's machine.

The architecture follows a five-stage pipeline: (1) *Parse* — PDF.js decompresses the PDF and extracts text items with coordinates; (2) *Analyze* — the layout engine segments pages into columns and blocks; (3) *Detect* — specialized detectors identify equations, gibberish, tables, figures, and citations; (4) *Extract* — text is assembled into reading order and rendered to Markdown; (5) *Render* — the resulting Markdown is presented for download alongside extracted images in a ZIP archive. This paper focuses primarily on stages 2 and 3: the document layout analysis engine proper.

2. SYSTEM ARCHITECTURE

LiteDoc is implemented as an immediately-invoked function expression (IIFE) in JavaScript, exposing a `LiteDoc` class with a `parse()` method. The core library, `litedoc-core.js`, comprises approximately 1,900 lines of pure JavaScript with no framework dependencies beyond the three runtime CDN libraries: PDF.js (v3+), Tesseract.js (v5+ with WASM), and JSZip for archive creation.

The processing pipeline executes in chunked batches of 10 pages per chunk to manage browser memory constraints. After each chunk, all page canvases are explicitly cleaned up (`canvas.width = 0; canvas.height = 0`) and PDF page references are released via `page.cleanup()`. This chunked architecture enables processing of arbitrarily large PDFs — documents exceeding 1,000 pages have been successfully processed — without browser tab crashes.

A shared offscreen `<canvas>` element with `willReadFrequently: true` is reused across all page renders to minimize GPU context creation overhead. The rendering scale for vector figure extraction is configurable at

three levels (1.5×, 2.0×, 2.5×), balancing image quality against memory consumption.

The pipeline integrates five optional feature modules — OCR, table detection, vector figure extraction, mathematical equation detection, and citation parsing — each toggleable at runtime through a settings UI. When OCR is set to auto-detect mode, the engine renders the first page and uses Tesseract's script detection to determine the document language before processing subsequent pages. For CJK PDFs, the engine supports Unicode CMap character mapping via a configurable `PDFJS_CMAP_URL`.

3. DOCUMENT LAYOUT ANALYSIS

3.1 Page Segmentation

The layout engine classifies each page into one of four geometric types: *Single-Column*, *Multi-Column*, *Landscape/Presentation*, or *Complex/Mixed-Grid*. Classification is determined before any content processing occurs.

A page is classified as Landscape if its width exceeds 105% of its height — appropriate for presentation slides and rotated tables. Otherwise, the engine performs vertical white-space analysis using X-interval projection. All text items on the page are projected onto the horizontal axis as intervals `[x, x + width]`. Intervals are sorted and merged with a tolerance of 0.5% of page width. Gaps between merged intervals represent vertical gutters — candidate column separators.

Each gutter is scored by its width minus a distance-from-center penalty. Gutters near the page center are preferred; those more than 35% from the center are heavily penalized to handle asymmetric magazine layouts (e.g., one-third/two-thirds splits). The highest-scoring valid gutter is selected. Zero gutters imply single-column layout; exactly one gutter indicates multi-column; multiple gutters signify a complex or mixed-grid layout.

Items spanning more than 55% of page width or crossing a gutter boundary are classified as spanning elements (full-width head-

ings, wide tables, figures) and processed separately from column content.

3.2 Reading Order Determination

Determining reading order in multi-column documents is fundamental to document layout analysis. Without correct ordering, column text becomes interleaved: lines from column B appear between lines from column A, rendering the extracted text unintelligible.

LiteDoc solves this problem using Kahn's (1962) topological sort algorithm applied to a directed acyclic graph (DAG) constructed from geometric bounding boxes. For each pair of blocks i and j , the algorithm establishes directional edges based on spatial heuristics:

Vertical ordering: Block i precedes block j if i is physically above j ($yMax_i \geq yMin_j - 10$ tolerance) and their horizontal projections overlap by at least 10 pixels or 10% of the smaller block's width. This constraint prevents blocks from different columns from being ordered vertically.

Horizontal ordering: Block i precedes block j if i is to the left of j ($xMax_i \leq xMin_j + 20$ tolerance) and they overlap vertically by at least 20 pixels. This handles intra-column ordering when blocks share the same vertical space.

The algorithm maintains an in-degree array and adjacency list. Zero-indegree nodes enter a priority queue sorted by a custom heuristic: blocks in distinct columns (no horizontal overlap) are ordered left-to-right by $xMin$; blocks within the same column are ordered top-to-bottom by $yMax$. Processing continues in Kahn's standard queue-based manner. If cycles exist (topologically impossible for spatial data but handled defensively), remaining unvisited blocks are appended to preserve completeness.

This geometric approach achieves correct reading order without any natural language processing or machine learning models — a key efficiency advantage for browser-based execution where loading neural models would impose unacceptable latency and memory costs.

3.3 Mathematical Equation Detection

Mathematical content in PDFs presents a unique challenge: symbols like $\int$, $\sum$, and ∂ are represented as individual glyphs with no semantic grouping, while inline math may be interspersed with body text at the same font size. LiteDoc employs a statistical Unicode symbol density heuristic for equation detection.

The detector scans each line group for a set of ~90 mathematical Unicode symbols spanning operators ($\sum$, $\int$, $\prod$, ∂ , $\sqrt{}$, ∞ , ∇), relations ($\approx$, $\neq$, $\equiv$, $\leq$, $\geq$), and Greek letters commonly used in mathematical notation (π , θ , λ , μ , σ , ϕ , ω , Δ , Ω , α , β , γ , and their variants). Common ASCII characters ($=$, $+$, $/$, $<$, $>$, $\wedge$, $_$) that appear in both mathematical and non-mathematical contexts are excluded from the symbol set to minimize false positives.

Three complementary rules determine equation classification. **High-density rule:** if `symbolCount` ≥ 2 and the symbol density `symbolCount / textLength` > 0.15 , the line is classified as mathematics. **Short-line rule:** if `symbolCount` ≥ 1 , text length is under 20 characters, and density exceeds 0.30, the line is classified as a standalone equation. **Assignment pattern rule:** lines matching the pattern `^[A-Z][a-z]?[s*=\s*]` (e.g., "E =", "Fg =") are classified as equation-mode assignments. Detected equations are wrapped in KaTeX-compatible `$$...$$` delimiters in the output Markdown and removed from the body text flow as garbage.

3.4 Gibberish Detection

PDF corruption — particularly in custom-encoded fonts — produces "gibberish" text: strings of nonsensical Unicode characters that pass through PDF.js text extraction but have no human-readable meaning. Common manifestations include random CJK ideographs in Latin-script documents (indicating corrupted CMap mappings), Private Use Area (PUA) characters, and mixed-script artifacts from font subsetting errors.

LiteDoc implements a two-tier gibberish detection system. The first tier performs a **corrupted font pre-scan** on the first three pages of each document. Text items are grouped by font name, and each font's suspicious character ratio is computed. The `scriptBucket()` function classifies each Unicode code point into 36 script categories, with 14 categories designated as suspicious (PUA, PUA supplementary, CJK unified, CJK radicals, CJK symbols, CJK compatibility, Hiragana, Katakana, enclosed CJK, Braille, Mongolian, Yi, miscellaneous symbols, control pictures, specials). Fonts exceeding 30% suspicious characters are hard-classified as corrupted; those above 8% suspicious characters that also contain Latin characters (indicating likely mixed-script corruption) are soft-classified.

The second tier computes a **per-item gibberish score** using the formula:

```
gibberishScore = min(1, puaRatio ×
    2.0 + suspRatio × 1.2 +
    mixPenalty)
```

where `puaRatio` is the fraction of Private Use Area characters, `suspRatio` is the fraction of suspicious-bucket characters, and `mixPenalty` is 0.5 if the item contains Latin characters alongside two or more suspicious script categories (otherwise 0). Items scoring above the configurable `garbageScoreThreshold` (default 0.20) are marked as garbage. Fonts previously flagged as corrupted result in immediate 1.0 scores for all their text items.

When a page is found to have a sufficiently high overall garbage ratio exceeding the `pageGarbageRatioThreshold` (default 0.60), processing falls back to OCR via Tesseract.js WASM for that page. This ensures that scanned or heavily corrupted pages still yield usable text.

3.5 Table Reconstruction

Table detection operates on the text grid reconstructed from spatial item positions. Items are first grouped into horizontal lines with an adaptive tolerance of 45% of the item's font height. Lines are then analyzed for columnar

structure: cells are formed by clustering items whose horizontal gaps are below 80% of font height.

Lines with two or more cells are candidate table rows. Consecutive table rows are grouped into table blocks; singleton rows break the block. For each block, column boundaries are computed by clustering cell starting positions rounded to 10-pixel bins, then consolidating columns closer than 25 pixels. The resulting grid is validated: blocks with fill ratios below 10% or overlap ratios above 60% (indicating paragraph-like text rather than tabular data) are rejected.

Merged cells are detected when a cell's right extent (`xMax`) extends significantly beyond its assigned column boundary, spanning into adjacent columns. Cross-page table continuations are detected by matching column headers between consecutive pages — if 50% or more of the header text matches the previous page's last table, the continuation is stitched without duplicating the header row. Output is formatted as GitHub-Flavored Markdown tables.

3.6 Vector Figure Extraction

Many PDFs contain figures — charts, diagrams, and illustrations — drawn as vector paths rather than embedded raster images. LiteDoc extracts these by analyzing the PDF operator list for vector drawing operations including `moveTo`, `lineTo`, `curveTo` (and its variants `curveTo2`, `curveTo3`), `closePath`, `endPath`, `paintPath`, `constructPath`, and `rectangle`. A page must contain at least 20 vector operations to trigger extraction, filtering out stray decorative lines.

Bounding boxes from `constructPath` and `rectangle` operations (with minimum dimensions of 2×2 pixels) are clustered using an overlap-based greedy algorithm with a padding of `max(20px, 2% of page width)`. Clusters with at least one constituent box, spanning at least 8% of page width and 5% of page height (and under 95% in either dimension to exclude full-page backgrounds), are classified as figures.

Each figure is rendered by first drawing the full page to an offscreen canvas at the configured scale ($1.5\times\text{--}2.5\times$), then cropping the figure's bounding box region with a 6-pixel margin, and exporting as JPEG at 82%–96% quality. Figures are injected into the text flow at the nearest line group by vertical proximity. Pages with more than 2,000 vector paths are skipped to avoid performance degradation from complex vector art.

3.7 Fingerprinting

Headers, footers, and page numbers — collectively termed "boilerplate" — are removed through frequency analysis. During a fingerprinting pass, the first 12 pages (or all pages if fewer) are processed to extract raw line text. Lines appearing on at least 50% of pages ($\text{Math.floor}(\text{totalPages} \times 0.5)$, minimum threshold of 2) are classified as repeating boilerplate. During the main processing pass, lines matching a fingerprint *and* positioned in the top 8% or bottom 8% of the page (configurable via `topMarginThreshold` and `bottomMarginThreshold`) are marked as garbage and excluded from output.

4. HYPERPARAMETER OPTIMIZATION

The DLA engine exposes 13 configurable parameters that govern text extraction behavior, listed in Table 1. These parameters control spatial tolerances for line grouping, block merging, margin filtering, table detection sensitivity, and gibberish scoring thresholds. Manual tuning of this parameter space is labor-intensive and prone to overfitting to specific document types.

Parameter	Default	Function
horizontalGapMultiplier	1.0	Space insertion gap scaling
rowSplitMultiplier	3.0	Vertical line-splitting tolerance
horizontalGapMin	12	Minimum gutter width (px)
bottomMarginThreshold	0.92	Bottom margin cutoff (fraction)
topMarginThreshold	0.08	Top margin cutoff (fraction)
distanceFromCenter-Penalty	3.0	Gutter center-distance weight
tableDensityThreshold	0.6	Minimum table fill ratio
pageGarbageRatio-Threshold	0.60	OCR fallback trigger ratio
paragraphGapThreshold	1.8	Paragraph break spacing
continuationGapThreshold	1.6	Line continuation spacing
tableGapYThreshold	3.5	Table row gap tolerance
itemOverlapTolerance	4	Duplicate item filter radius (px)
garbageScoreThreshold	0.20	Gibberish score cutoff

Table 1. Configurable hyperparameters and their default values.

We employed an automated hyperparameter optimization framework using genetic algorithms with holdout validation. The optimization pipeline operates on a synthetic benchmark dataset of 2,054 PDFs spanning seven document types — Academic papers, Magazines, Books, intentionally corrupted documents ("Cursed"), Research Papers, Math Papers, and Notebooks — each paired with ground-truth Markdown.

The genetic algorithm initializes a diversified population combining elite configurations (previous best candidates), small random perturbations of elite configurations, and purely random candidates to maintain exploration diversity. Tournament selection with $k = 3$ is used for parent selection. Each candidate's fitness is computed as the mean-plus-median of per-file fuzzy match scores against ground truth, evaluated on the training portion (80%) of the dataset.

Holdout validation gates all deployment decisions: 20% of the dataset is reserved and never seen during training. After each epoch, the population's best candidate is evaluated on the holdout set, and only configurations exceeding a 95% holdout pass rate are eligible for deployment. This prevents the optimizer from overfitting to the training distribution.

Plateau detection with adaptive restart prevents local minima entrapment. If the best fitness fails to improve for three consecutive epochs, the mutation rate is increased and the population is re-initialized at a larger size. A per-epoch annealing reset further prevents the population from converging prematurely on suboptimal regions of the parameter space.

Scoring employs a proportional penalty mechanism: output Markdown is compared to ground truth using fuzzy word overlap (Jaccard similarity) and structural element detection (tables, math blocks, lists, and headers). This multi-metric approach ensures that improvements in one dimension (e.g., word accuracy) are not achieved at the expense of another (e.g., table structure preservation).

Note: The optimization framework and synthetic benchmark dataset are internal research tools not included in the public LiteDoc repository. GA-tuned parameters are deployed in v2.6.0 via `assets/datasets/config.json`.

5. EVALUATION

Benchmark Categories

- **Academic** — Single-column formal papers with abstracts, author affiliations, and bibliographic references.
- **Research Paper** — Two-column IEEE/ACM format with dense text, floating figures, and numbered equations.
- **Math Paper** — Heavy mathematical content: inline/displayed equations, theorem-proof environments, Greek/Unicode symbols.
- **Book** — Long-form prose with chapters, consistent margins, and extended paragraph runs.
- **Magazine** — Multi-column layouts with embedded images, pull quotes, and varied

typography.

- **Notebook** — Mixed code blocks, Markdown headers, inline LaTeX, and narrative prose (Jupyter/Colab style).
- **Cursed** — Intentionally corrupted PDFs: PUA characters, mixed-script artifacts, encoding errors — tests gibberish detection.

The synthetic benchmark comprises seven document categories designed to stress-test different aspects of the DLA pipeline:

- **Academic** — Single-column papers with headers, footnotes, and reference sections.
- **Magazine** — Multi-column layouts with embedded images, pull quotes, and varied typography.
- **Book** — Long-form prose with chapters, consistent margins, and minimal structural variation.
- **Cursed** — Intentionally corrupted PDFs with encoding errors, PUA characters, and mixed-script artifacts to test gibberish detection robustness.
- **Research Paper** — Two-column IEEE/ACM format papers with figures, tables, equations, and citations.
- **Math Paper** — Heavy mathematical content with inline equations, displayed equations, and theorem/proof environments.
- **Notebook** — Mixed content: code blocks, Markdown-style headers, LaTeX math, and prose.

The total dataset contains 2,054 PDFs, each with ground-truth Markdown used for fuzzy scoring. The fuzzy scoring system evaluates word overlap via Jaccard similarity on normalized token sets and separately scores structural elements: table detection (row/column count accuracy), math block preservation, list and heading detection.

A matched A/B comparison was conducted on 77 identical PDFs (all 7 document types). Both v2.5.2 (previous release) and v2.6.0 (bug-fixed release) were run on identical PDFs with expected-string match scoring:

Metric	v2.5.2	v2.6.0	Δ
Avg match score	52.1%	56.9%	+4.8%
Pass rate ($\geq 70\%$)	26% (20/77)	36.4% (28/77)	+10pp

The GA optimizer framework (tournament selection, holdout validation, plateau detection) is fully built and deployed with 13 tunable hyperparameters targeting the 2,054-PDF synthetic dataset. Full training convergence and optimized hyperparameter deployment are scheduled for v3.0. The v2.6.0 improvements documented above derive from six source-level bug fixes (see CHANGELOG), not from GA tuning.

6. RELATED WORK

PDF extraction libraries. pdfplumber (Singer-Vine, 2016) provides Python-based PDF text and table extraction with meticulous character-level positioning but requires a server-side Python environment. Camelot (Mehta et al., 2019) specializes in table extraction from PDFs using lattice and stream methods. Tabula (Aristarán et al., 2014) offers a graphical interface for table extraction. All three operate on the extraction side of a client-server boundary.

Academic document processing. GROBID (Lopez, 2009) uses conditional random fields for bibliographic data and full-text extraction from scientific PDFs but requires Java runtime and substantial memory. Science Parse (Allen Institute for AI, 2018) extracts title, authors, sections, and references from academic papers. ScholarPhi (Head et al., 2021) augments PDF reading with term definitions and citation context but is not a general-purpose extraction tool.

Commercial solutions. Adobe Extract API provides cloud-based structured content extraction with high accuracy for enterprise documents. Amazon Textract offers OCR and table extraction at per-page pricing. Senseforth and similar services provide managed document processing pipelines. All require server-side transmission of document content.

Key differentiator. LiteDoc distinguishes itself from all categories above by executing entirely within the browser, with no server dependency, no API keys, no per-document costs, and no data exfiltration. The combination of geometric reading order determination via Kahn's algorithm, statistical math detection via Unicode symbol density, and evolutionary hyperparameter optimization is — to our knowledge — unique among open-source PDF extraction tools.

7. CONCLUSION

This paper has presented LiteDoc, a browser-native document layout analysis engine that extracts structured Markdown from PDF files with 100% local execution. The engine demonstrates that fully client-side DLA is viable for academic and general-purpose documents, achieving correct multi-column reading order through Kahn's topological sort on geometric block graphs without relying on machine learning models or server infrastructure.

Statistical heuristics for mathematical equation detection and gibberish filtering perform robustly on the synthetic benchmark, and the evolutionary hyperparameter optimization framework provides measurable improvement over manual parameter configuration. The genetic algorithm's use of holdout validation ensures that optimized parameters generalize beyond the training distribution.

Future work. Several directions warrant further investigation. Table structure benchmarks with standardized metrics (e.g., TEDS for table similarity) would enable rigorous comparison with dedicated table extraction tools. Right-to-left language support for Arabic, Hebrew, and Persian documents requires deeper integration with bidirectional text algorithms. Packaging LiteDoc as Python (via Pyodide) and npm modules would expand its integration surface for server-side use cases while preserving the option of local execution. Finally, extending the synthetic benchmark to include multilingual and non-Latin script doc-

uments would broaden the scope of validated extraction quality.

REFERENCES

- Aristarán, M., Tigas, M., & Merrill, J. B. (2014). Tabula: Extract tables from PDFs. *Journal of Open Source Software*.
- Eisenberg, B., & Sharif, A. (2014). KaTeX: The fastest math typesetting library for the web. *GitHub Repository*. <https://github.com/KaTeX/KaTeX>
- Head, A., Lo, K., Kang, D., Fok, R., Skjonsberg, S., Weld, D. S., & Hearst, M. A. (2021). Augmenting scientific papers with just-in-time, position-sensitive definitions of terms and symbols. *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, 1–18. <https://doi.org/10.1145/3411764.3445648>
- Holland, J. H. (1975). *Adaptation in natural and artificial systems*. University of Michigan Press.
- Kahn, A. B. (1962). Topological sorting of large networks. *Communications of the ACM*, 5(11), 558–562. <https://doi.org/10.1145/368996.369025>
- Lopez, P. (2009). GROBID: Combining automatic bibliographic data recognition and term extraction for scholarship publications. *Proceedings of the 13th European Conference on Research and Advanced Technology for Digital Libraries*, 473–474.
- Mehta, V., Bhardwaj, A., Singh, M., & Iyengar, S. (2019). Camelot: A Python library for PDF table extraction. *Journal of Open Source Software*, 4(37), 1286.
- Mozilla Corporation. (2023). PDF.js: A general-purpose, web standards-based platform for parsing and rendering PDFs. *GitHub Repository*. <https://github.com/mozilla/pdf.js>
- Dutschke, S. (2009). JSZip: Create, read and edit .zip files with JavaScript. *GitHub Repository*. <https://github.com/Stuk/jszip>
- Singer-Vine, J. (2016). pdfplumber: Plumb a PDF for detailed information about each text character, rectangle, and line. *GitHub Repository*. <https://github.com/jsvine/pdfplumber>
- Smith, R. (2007). An overview of the Tesseract OCR engine. *Proceedings of the Ninth International Conference on Document Analysis and Recognition*, 2, 629–633. <https://doi.org/10.1109/ICDAR.2007.4376991>